GitHub Advanced Security Full Report

GitHub Repository: vishutyagi0210/c--vol-scanning

Generated: 2026-06-01T09:57:44.942Z

Software Composition Analysis Summary

Dependencies 0

Unprocessed manifests 0 | Processed manifests 0

Open Dependency Vulnerabilities 0

High 0 | Warning 0 | Moderate 0 | Low 0

Code Scanning Summary

Open Findings 19

Critical 5 | High 8 | Medium 3 | Low 0

Error 0 | Warning 0 | Note 3

Closed Findings 0

No Results 0

Software Composition Analysis - Manifests

Processed Manifest Files 0

Name

Software Composition Analysis Vulnerabilities

Code Scanning Vulnerabilities

Code Scanning Rules Applied 164

Rule	Precision	Severity	Tags
cs/web/file-upload	high	recommendation	security,maintainability,frameworks/asp.net,external/cwe/cwe-434
cs/ecb-encryption	high	warning	security,external/cwe/cwe-327
cs/xml/xpath-injection	high	error	security,external/cwe/cwe-643
cs/web/debug-binary	very-high	warning	security,maintainability,frameworks/asp.net,external/cwe/cwe-011,external/cwe/cwe-532
cs/resource-injection	high	error	security,external/cwe/cwe-099
cs/ldap-injection	high	error	security,external/cwe/cwe-090
cs/unvalidated-local-pointer-arithmetic	high	warning	security,external/cwe/cwe-119,external/cwe/cwe-120,external/cwe/cwe-122,external/cwe/cwe-788
cs/assembly-path-injection	high	error	security,external/cwe/cwe-114
cs/uncontrolled-format-string	high	error	security,external/cwe/cwe-134
cs/regex-injection	high	error	security,external/cwe/cwe-730,external/cwe/cwe-400
cs/redos	high	error	security,external/cwe/cwe-1333,external/cwe/cwe-730,external/cwe/cwe-400
cs/web/persistent-cookie	high	warning	security,external/cwe/cwe-539
cs/web/request-validation-disabled	high	warning	security,frameworks/asp.net,external/cwe/cwe-016
cs/web/broad-cookie-domain	high	warning	security,external/cwe/cwe-287
cs/inadequate-rsa-padding	high	warning	security,external/cwe/cwe-327,external/cwe/cwe-780
cs/xml/insecure-dtd-handling	high	error	security,external/cwe/cwe-611,external/cwe/cwe-827,external/cwe/cwe-776
cs/web/missing-global-error-handler	high	warning	security,external/cwe/cwe-012,external/cwe/cwe-248
cs/web/missing-token-validation	high	error	security,external/cwe/cwe-352
cs/exposure-of-sensitive-information	high	error	security,external/cwe/cwe-359
cs/web/cookie-httponly-not-set	high	warning	security,external/cwe/cwe-1004
cs/xml/missing-validation	high	recommendation	security,external/cwe/cwe-112
cs/insecure-randomness	high	warning	security,external/cwe/cwe-338
cs/code-injection	high	error	security,external/cwe/cwe-094,external/cwe/cwe-095,external/cwe/cwe-096

Rule	Precision	Severity	Tags
cs/xml-injection	high	error	security,external/cwe/cwe-091
cs/log-forging	high	error	security,external/cwe/cwe-117
cs/web/directory-browse-enabled	very-high	warning	security,external/cwe/cwe-548
cs/web/xss	high	error	security,external/cwe/cwe-079,external/cwe/cwe-116
cs/sensitive-data-transmission	high	error	security,external/cwe/cwe-201
cs/weak-encryption	high	warning	security,external/cwe/cwe-327
cs/information-exposure-through-exception	high	error	security,external/cwe/cwe-209,external/cwe/cwe-497
cs/user-controlled-bypass	high	error	security,external/cwe/cwe-807,external/cwe/cwe-247,external/cwe/cwe-350
cs/command-line-injection	high	error	correctness,security,external/cwe/cwe-078,external/cwe/cwe-088
cs/web/disabled-header-checking	high	warning	security,external/cwe/cwe-113
cs/insufficient-key-size	high	warning	security,external/cwe/cwe-326
cs/sql-injection	high	error	security,external/cwe/cwe-089
cs/cleartext-storage-of-sensitive-information	high	error	security,external/cwe/cwe-312,external/cwe/cwe-315,external/cwe/cwe-359
cs/web/broad-cookie-path	high	warning	security,external/cwe/cwe-287
cs/web/cookie-secure-not-set	high	error	security,external/cwe/cwe-319,external/cwe/cwe-614
cs/web/requiressl-not-set	high	error	security,external/cwe/cwe-319,external/cwe/cwe-614
cs/deserialized-delegate	high	warning	security,external/cwe/cwe-502
cs/unsafe-deserialization-untrusted-input	high	error	security,external/cwe/cwe-502
cs/web/unvalidated-url-redirection	high	error	security,external/cwe/cwe-601
cs/web/missing-x-frame-options	high	error	security,external/cwe/cwe-451,external/cwe/cwe-829
cs/session-reuse	high	error	security,external/cwe/cwe-384
cs/path-injection	high	error	security,external/cwe/cwe-022,external/cwe/cwe-023,external/cwe/cwe-036,external/cwe/cwe-073,external/cwe/cwe-099
cs/zipslip	high	error	security,external/cwe/cwe-022
cs/web/ambiguous-server-variable	medium	warning	security,maintainability,frameworks/asp.net,external/cwe/cwe-348
cs/web/ambiguous-client-variable	medium	warning	security,maintainability,frameworks/asp.net,external/cwe/cwe-348
cs/insecure-sql-connection	medium	error	security,external/cwe/cwe-327

Rule	Precision	Severity	Tags
cs/web/missing-function-level-access-control	medium	warning	security,external/cwe/cwe-285,external/cwe/cwe-284,external/cwe/cwe-862
cs/web/insecure-direct-object-reference	medium	warning	security,external/cwe/cwe-639
cs/serialization-check-bypass	medium	warning	security,external/cwe/cwe-020
cs/empty-password-in-configuration	medium	warning	security,external/cwe/cwe-258,external/cwe/cwe-862
cs/thread-unsafe-icryptotransform-field-in-class	medium	warning	concurrency,security,external/cwe/cwe-362
cs/thread-unsafe-icryptotransform-captured-in-lambda	medium	warning	concurrency,security,external/cwe/cwe-362
cs/summary/lines-of-code			summary,lines-of-code,debug
cs/telemetry/supported-external-api			summary,telemetry
cs/telemetry/supported-external-api-sources			summary,telemetry
cs/telemetry/extraction-information			summary,telemetry
cs/telemetry/supported-external-api-taint			summary,telemetry
cs/telemetry/external-libs			summary,telemetry
cs/telemetry/unsupported-external-api			summary,telemetry
cs/telemetry/supported-external-api-sinks			summary,telemetry
cs/coupled-types	high	recommendation	maintainability,complexity,modularity
cs/linq/missed-where	high	recommendation	quality,maintainability,readability,language-features
cs/linq/missed-oftype	high	recommendation	quality,maintainability,readability,language-features
cs/linq/missed-select	high	recommendation	quality,maintainability,readability,language-features
cs/linq/inconsistent-enumeration	medium	warning	quality,reliability,correctness,external/cwe/cwe-834
cs/linq/missed-cast	high	recommendation	quality,maintainability,readability,language-features
cs/linq/useless-select	very-high	warning	quality,maintainability,useless-code,language-features,external/cwe/cwe-561
cs/linq/missed-all	high	recommendation	quality,maintainability,readability,language-features
cs/string-concatenation-in-loop	very-high	recommendation	quality,reliability,performance
cs/inefficient-containskey	high	recommendation	quality,reliability,performance

Rule	Precision	Severity	Tags
cs/stringbuilder-creation-in-loop	very-high	recommendation	quality,reliability,performance
cs/unmanaged-code	high	recommendation	quality,reliability,correctness
cs/constant-condition	very-high	warning	quality,maintainability,readability,external/cwe/cwe-835
cs/too-many-ref-parameters	very-high	recommendation	maintainability,readability,testability
cs/local-shadows-member	high	recommendation	quality,maintainability,readability
cs/virtual-call-in-constructor	medium	warning	quality,reliability,correctness
cs/path-combine	very-high	recommendation	quality,reliability,correctness
cs/call-to-unmanaged-code	high	recommendation	quality,reliability,correctness
cs/catch-nullreferenceexception	very-high	warning	quality,reliability,correctness,error-handling,external/cwe/cwe-395
cs/class-name-comparison	medium	warning	quality,reliability,correctness,external/cwe/cwe-486
cs/field-masks-base-field	high	warning	quality,maintainability,readability,naming
cs/class-name-matches-base-class	high	recommendation	quality,maintainability,readability,naming
cs/expose-implementation	high	recommendation	quality,reliability,correctness,external/cwe/cwe-485
cs/cast-from-abstract-to-concrete-collection	medium	warning	quality,reliability,correctness,external/cwe/cwe-485
cs/empty-catch-block	very-high	recommendation	quality,reliability,error-handling,exceptions,external/cwe/cwe-390,external/cwe/cwe-391
cs/asp/response-write	high	recommendation	quality,maintainability,readability,frameworks/asp.net
cs/wrong-equals-signature	medium	warning	quality,reliability,correctness
cs/invalid-string-formatting	high	error	quality,reliability,correctness
cs/dispose-not-called-on-throw	medium	warning	quality,reliability,error-handling,performance,external/cwe/cwe-404,external/cwe/cwe-459,external/cwe/cwe-460
cs/null-argument-to-equals	high	warning	quality,reliability,correctness
cs/class-missing-equals	medium	error	quality,reliability,correctness
cs/local-not-disposed	high	warning	quality,reliability,correctness,efficiency,external/cwe/cwe-404,external/cwe/cwe-459,external/cwe/cwe-460
cs/wrong-compareto-signature	medium	warning	quality,reliability,correctness
cs/call-to-obsolete-method	very-high	warning	quality,maintainability,changeability,external/cwe/cwe-477
cs/class-implements-icloneable	very-high	recommendation	quality,maintainability
cs/call-to-gc	very-high	warning	quality,reliability,performance
cs/inconsistent-equals-and-gethashcode	medium	warning	quality,reliability,correctness,external/cwe/cwe-581

Rule	Precision	Severity	Tags
cs/complex-condition	high	recommendation	maintainability,readability,testability
cs/complex-block	high	recommendation	maintainability,complexity,testability
cs/xmldoc/missing-summary	high	recommendation	maintainability,readability
cs/comparison-of-identical-expressions	high	warning	quality,reliability,correctness
cs/dereferenced-value-is-always-null	very-high	error	quality,reliability,correctness,exceptions,external/cwe/cwe-476
cs/dereferenced-value-may-be-null	high	warning	quality,reliability,correctness,exceptions,external/cwe/cwe-476
cs/useless-type-test	medium	warning	quality,maintainability,useless-code,external/cwe/cwe-561
cs/nested-if-statements	high	recommendation	quality,maintainability,readability,language-features
cs/missed-ternary-operator	high	recommendation	quality,maintainability,readability,language-features
cs/rethrown-exception-variable	very-high	warning	quality,reliability,error-handling,language-features
cs/missed-readonly-modifier	high	recommendation	quality,maintainability,readability,language-features
cs/missed-using-statement	high	recommendation	quality,maintainability,readability,language-features
cs/coalesce-of-identical-expressions	medium	error	quality,maintainability,useless-code,external/cwe/cwe-561
cs/useless-upcast	medium	warning	quality,maintainability,useless-code,external/cwe/cwe-561
cs/catch-of-all-exceptions	high	recommendation	quality,reliability,error-handling,external/cwe/cwe-396
cs/type-test-of-this	high	warning	quality,reliability,correctness,testability,language-features
cs/simplifiable-boolean-expression	high	recommendation	quality,maintainability,readability
cs/cast-of-this-to-type-parameter	high	recommendation	quality,reliability,correctness,language-features
cs/useless-cast-to-self	medium	warning	quality,maintainability,useless-code,external/cwe/cwe-561
cs/downcast-of-this	high	warning	quality,reliability,correctness,testability,language-features
cs/chained-type-tests	high	recommendation	reliability,performance,changeability,language-features
cs/unused-property-value	high	warning	quality,reliability,correctness,language-features
cs/lock-this	high	warning	quality,reliability,concurrency,modularity,external/cwe/cwe-662
cs/unsynchronized-static-access	medium	error	quality,reliability,concurrency,external/cwe/cwe-362,external/cwe/cwe-567
cs/unsynchronized-getter	medium	error	quality,reliability,concurrency,correctness,external/cwe/cwe-662
cs/inconsistent-lock-sequence	high	error	quality,reliability,concurrency,correctness,external/cwe/cwe-662
cs/unsafe-double-checked-lock	medium	error	quality,reliability,concurrency,external/cwe/cwe-609

Rule	Precision	Severity	Tags
cs/locked-wait	high	warning	quality,reliability,concurrency,correctness,external/cwe/cwe-662,external/cwe/cwe-833
cs/unsafe-sync-on-field	high	error	quality,reliability,concurrency,correctness,external/cwe/cwe-662,external/cwe/cwe-366
cs/equals-on-unrelated-types	high	error	quality,reliability,correctness
cs/non-short-circuit	high	error	quality,reliability,correctness,external/cwe/cwe-480,external/cwe/cwe-691
cs/empty-block	high	warning	quality,reliability,correctness
cs/misleading-indentation	medium	warning	quality,maintainability,readability,correctness
cs/empty-lock-statement	high	warning	quality,reliability,concurrency,changeability,language-features,external/cwe/cwe-585
cs/self-assignment	high	error	quality,reliability,correctness
cs/equals-uses-as	medium	warning	quality,reliability,correctness
cs/equality-on-floats	high	warning	quality,reliability,correctness
cs/equals-on-arrays	high	recommendation	quality,reliability,correctness
cs/mishandling-japanese-era	medium	warning	quality,reliability,correctness
cs/loss-of-precision	high	error	quality,reliability,correctness,external/cwe/cwe-190,external/cwe/cwe-192,external/cwe/cwe-197,external/cwe/cwe-681
cs/impossible-array-cast	high	error	quality,reliability,correctness
cs/nested-loops-with-same-variable	high	warning	quality,reliability,correctness
cs/reference-equality-on-valuetypes	high	error	quality,reliability,correctness,external/cwe/cwe-595
cs/gethashcode-is-not-defined	high	warning	quality,reliability,correctness
cs/reference-equality-with-object	medium	warning	quality,reliability,correctness,external/cwe/cwe-595
cs/equals-uses-is	medium	warning	quality,reliability,correctness
cs/invalid-dynamic-call	medium	error	quality,reliability,correctness,external/cwe/cwe-628
cs/recursive-equals-call	high	error	quality,reliability,correctness
cs/inconsistent-compareto-and-equals	medium	warning	quality,reliability,correctness
cs/recursive-operator-equals-call	medium	error	quality,reliability,correctness
cs/empty-collection	high	error	quality,reliability,correctness,external/cwe/cwe-561
cs/index-out-of-bounds	high	error	quality,reliability,correctness,external/cwe/cwe-193
cs/unused-collection	high	error	quality,maintainability,useless-code,external/cwe/cwe-561

Rule	Precision	Severity	Tags
cs/test-for-negative-container-size	very-high	warning	quality,reliability,correctness
cs/unsafe-year-construction	medium	warning	quality,reliability,correctness
cs/unchecked-cast-in-equals	high	warning	quality,reliability,correctness
cs/static-field-written-by-instance	high	recommendation	quality,maintainability,modularity
cs/stringbuilder-initialized-with-character	high	error	quality,reliability,correctness
cs/useless-assignment-to-local	very-high	warning	quality,maintainability,useless-code,external/cwe/cwe-563
cs/useless-tostring-call	high	recommendation	quality,maintainability,useless-code
cs/useless-gethashcode-call	high	recommendation	quality,maintainability,readability,useless-code
cs/useless-if-statement	very-high	warning	quality,maintainability,readability,useless-code
cs/call-to-object-tostring	very-high	warning	quality,reliability,correctness
cs/unused-label	high	warning	quality,maintainability,useless-code

CWE Coverage

101

cwe-011 cwe-012 cwe-016 cwe-020 cwe-022 cwe-023 cwe-036 cwe-073 cwe-078 cwe-079 cwe-088 cwe-089 cwe-090
cwe-091 cwe-094 cwe-095 cwe-096 cwe-099 cwe-1004 cwe-112 cwe-113 cwe-114 cwe-116 cwe-117 cwe-119 cwe-120
cwe-122 cwe-1333 cwe-134 cwe-190 cwe-192 cwe-193 cwe-197 cwe-201 cwe-209 cwe-247 cwe-248 cwe-258 cwe-284
cwe-285 cwe-287 cwe-312 cwe-315 cwe-319 cwe-326 cwe-327 cwe-338 cwe-348 cwe-350 cwe-352 cwe-359 cwe-362
cwe-366 cwe-384 cwe-390 cwe-391 cwe-395 cwe-396 cwe-400 cwe-404 cwe-434 cwe-451 cwe-459 cwe-460 cwe-476
cwe-477 cwe-480 cwe-485 cwe-486 cwe-497 cwe-502 cwe-532 cwe-539 cwe-548 cwe-561 cwe-563 cwe-567 cwe-581
cwe-585 cwe-595 cwe-601 cwe-609 cwe-611 cwe-614 cwe-628 cwe-639 cwe-643 cwe-662 cwe-681 cwe-691 cwe-730
cwe-776 cwe-780 cwe-788 cwe-807 cwe-827 cwe-829 cwe-833 cwe-834 cwe-835 cwe-862

Open Vulnerabilities

Critical 5

Rule	Found on	Discovered by
Deserialization of untrusted data	2026-06-01T09:03:22Z	CodeQL
Deserialization of untrusted data	2026-06-01T09:03:22Z	CodeQL
Uncontrolled command line	2026-06-01T09:03:22Z	CodeQL
Uncontrolled command line	2026-06-01T09:03:22Z	CodeQL
Untrusted XML is read insecurely	2026-06-01T09:03:22Z	CodeQL

cs/unsafe-deserialization-untrusted-input

Location: VulnerableApi/Controllers/DeserializationController.cs:21

Commit SHA: 90619238a29bec2d4f15d6bacdfa555db078de43

Message: User-provided data flows to unsafe deserializer.

Description: Calling an unsafe deserializer with data controlled by an attacker can lead to denial of service and other security problems.

Precision: high

Severity: error

cwe-502

Deserialization of untrusted data

Deserializing an object from untrusted input may result in security problems, such as denial of service or remote code execution.

Note that a deserialization method is only dangerous if it can instantiate arbitrary classes. Serialization frameworks that use a schema to instantiate only expected, predefined types are generally not tracked by this query. Such frameworks are generally safe with respect to arbitrary-class-instantiation and gadget-chain attacks when the schema is trusted and does not permit user-controlled type resolution. However, care must be taken to ensure the schema strictly limits the allowed types. Permitting common standard library classes can still leave the application vulnerable to gadget-chain attacks.

Recommendation

Avoid deserializing objects from an untrusted source, and if not possible, make sure to use a safe deserialization framework.

Example

In this example, text from an HTML text box is deserialized using a `JavaScriptSerializer` with a simple type resolver. Using a type resolver means that arbitrary code may be executed.

```
using System.Web.UI.WebControls;
using System.Web.Script.Serialization;

class Bad
{
  public static object Deserialize(TextBox textBox)
  {
      JavaScriptSerializer sr = new JavaScriptSerializer(new SimpleTypeResolver());
      // BAD
      return sr.DeserializeObject(textBox.Text);
  }
}
```

To fix this specific vulnerability, we avoid using a type resolver. In other cases, it may be necessary to use a different deserialization framework.

```
using System.Web.UI.WebControls;
using System.Web.Script.Serialization;

class Good
{
  public static object Deserialize(TextBox textBox)
  {
      JavaScriptSerializer sr = new JavaScriptSerializer();
      // GOOD: no unsafe type resolver
      return sr.DeserializeObject(textBox.Text);
  }
}
```

In the following example potentially untrusted stream and type is deserialized using a `DataContractJsonSerializer` which is known to be vulnerable with user supplied types.

```
using System.Runtime.Serialization.Json;
using System.IO;
using System;

class BadDataContractJsonSerializer
{
  public static object Deserialize(string type, Stream s)
  {
      // BAD: stream and type are potentially untrusted
      var ds = new DataContractJsonSerializer(Type.GetType(type));
      return ds.ReadObject(s);
  }
}
```

To fix this specific vulnerability, we are using hardcoded Plain Old CLR Object (POCO) type. In other cases, it may be necessary to use a different deserialization framework.

```
using System.Runtime.Serialization.Json;
using System.IO;
using System;

class Poco
{
  public int Count;

  public string Comment;
}

class GoodDataContractJsonSerializer
{
  public static Poco Deserialize(Stream s)
  {
      // GOOD: while stream is potentially untrusted, the instantiated type is hardcoded
      var ds = new DataContractJsonSerializer(typeof(Poco));
      return (Poco)ds.ReadObject(s);
  }
}
```

References

- Muñoz, Alvaro and Mirosh, Oleksandr: JSON Attacks.
- Common Weakness Enumeration: CWE-502.

cs/unsafe-deserialization-untrusted-input

Location: VulnerableApi/Controllers/DeserializationController.cs:30

Commit SHA: 90619238a29bec2d4f15d6bacdfa555db078de43

Message: User-provided data flows to unsafe deserializer.

Description: Calling an unsafe deserializer with data controlled by an attacker can lead to denial of service and other security problems.

Precision: high

Severity: error

cwe-502

Deserialization of untrusted data

Deserializing an object from untrusted input may result in security problems, such as denial of service or remote code execution.

Note that a deserialization method is only dangerous if it can instantiate arbitrary classes. Serialization frameworks that use a schema to instantiate only expected, predefined types are generally not tracked by this query. Such frameworks are generally safe with respect to arbitrary-class-instantiation and gadget-chain attacks when the schema is trusted and does not permit user-controlled type resolution. However, care must be taken to ensure the schema strictly limits the allowed types. Permitting common standard library classes can still leave the application vulnerable to gadget-chain attacks.

Recommendation

Avoid deserializing objects from an untrusted source, and if not possible, make sure to use a safe deserialization framework.

Example

In this example, text from an HTML text box is deserialized using a `JavaScriptSerializer` with a simple type resolver. Using a type resolver means that arbitrary code may be executed.

```
using System.Web.UI.WebControls;
using System.Web.Script.Serialization;

class Bad
{
  public static object Deserialize(TextBox textBox)
  {
      JavaScriptSerializer sr = new JavaScriptSerializer(new SimpleTypeResolver());
      // BAD
      return sr.DeserializeObject(textBox.Text);
  }
}
```

To fix this specific vulnerability, we avoid using a type resolver. In other cases, it may be necessary to use a different deserialization framework.

```
using System.Web.UI.WebControls;
using System.Web.Script.Serialization;

class Good
{
  public static object Deserialize(TextBox textBox)
  {
      JavaScriptSerializer sr = new JavaScriptSerializer();
      // GOOD: no unsafe type resolver
      return sr.DeserializeObject(textBox.Text);
  }
}
```

In the following example potentially untrusted stream and type is deserialized using a `DataContractJsonSerializer` which is known to be vulnerable with user supplied types.

```
using System.Runtime.Serialization.Json;
using System.IO;
using System;

class BadDataContractJsonSerializer
{
  public static object Deserialize(string type, Stream s)
  {
      // BAD: stream and type are potentially untrusted
      var ds = new DataContractJsonSerializer(Type.GetType(type));
      return ds.ReadObject(s);
  }
}
```

To fix this specific vulnerability, we are using hardcoded Plain Old CLR Object (POCO) type. In other cases, it may be necessary to use a different deserialization framework.

```
using System.Runtime.Serialization.Json;
using System.IO;
using System;

class Poco
{
  public int Count;

  public string Comment;
}

class GoodDataContractJsonSerializer
{
  public static Poco Deserialize(Stream s)
  {
      // GOOD: while stream is potentially untrusted, the instantiated type is hardcoded
      var ds = new DataContractJsonSerializer(typeof(Poco));
      return (Poco)ds.ReadObject(s);
  }
}
```

References

- Muñoz, Alvaro and Mirosh, Oleksandr: JSON Attacks.
- Common Weakness Enumeration: CWE-502.

cs/command-line-injection

Location: VulnerableApi/Controllers/CommandInjectionController.cs:33

Commit SHA: 90619238a29bec2d4f15d6bacdfa555db078de43

Message: This command line depends on a user-provided value.

Description: Using externally controlled strings in a command line may allow a malicious user to change the meaning of the command.

Precision: high

Severity: error

cwe-078,cwe-088

Uncontrolled command line

Code that passes user input directly to `System.Diagnostic.Process.Start`, or some other library routine that executes a command, allows the user to execute malicious code.

Recommendation

If possible, use hard-coded string literals to specify the command to run or library to load. Instead of passing the user input directly to the process or library function, examine the user input and then choose among hard-coded string literals.

If the applicable libraries or commands cannot be determined at compile time, then add code to verify that the user input string is safe before using it.

Example

The following example shows code that takes a shell script that can be changed maliciously by a user, and passes it straight to `System.Diagnostic.Process.Start` without examining it first.

```
using System;
using System.Web;
using System.Diagnostics;

public class CommandInjectionHandler : IHttpHandler
{
  public void ProcessRequest(HttpContext ctx)
  {
      string param = ctx.Request.QueryString["param"];
      Process.Start("process.exe", "/c " + param);
  }
}
```

References

- OWASP: Command Injection.
- Common Weakness Enumeration: CWE-78.
- Common Weakness Enumeration: CWE-88.

cs/command-line-injection

Location: VulnerableApi/Controllers/CommandInjectionController.cs:17

Commit SHA: 90619238a29bec2d4f15d6bacdfa555db078de43

Message: This command line depends on a user-provided value.

Description: Using externally controlled strings in a command line may allow a malicious user to change the meaning of the command.

Precision: high

Severity: error

cwe-078,cwe-088

Uncontrolled command line

Code that passes user input directly to `System.Diagnostic.Process.Start`, or some other library routine that executes a command, allows the user to execute malicious code.

Recommendation

If possible, use hard-coded string literals to specify the command to run or library to load. Instead of passing the user input directly to the process or library function, examine the user input and then choose among hard-coded string literals.

If the applicable libraries or commands cannot be determined at compile time, then add code to verify that the user input string is safe before using it.

Example

The following example shows code that takes a shell script that can be changed maliciously by a user, and passes it straight to `System.Diagnostic.Process.Start` without examining it first.

```
using System;
using System.Web;
using System.Diagnostics;

public class CommandInjectionHandler : IHttpHandler
{
  public void ProcessRequest(HttpContext ctx)
  {
      string param = ctx.Request.QueryString["param"];
      Process.Start("process.exe", "/c " + param);
  }
}
```

References

- OWASP: Command Injection.
- Common Weakness Enumeration: CWE-78.
- Common Weakness Enumeration: CWE-88.

cs/xml/insecure-dtd-handling

Location: VulnerableApi/Controllers/XxeController.cs:21

Commit SHA: 90619238a29bec2d4f15d6bacdfa555db078de43

Message: This insecure XML processing depends on a user-provided value (DTD processing enabled in settings, insecure resolver set in settings).

Description: Untrusted XML is read with an insecure resolver and DTD processing enabled.

Precision: high

Severity: error

cwe-611,cwe-776,cwe-827

Untrusted XML is read insecurely

XML documents can contain Document Type Definitions (DTDs), which may define new XML entities. These can be used to perform Denial of Service (DoS) attacks, or resolve to resources outside the intended sphere of control.

Recommendation

When processing XML documents, ensure that DTD processing is disabled unless absolutely necessary, and if it is necessary, ensure that a secure resolver is used.

Example

The following example shows an HTTP request parameter being read directly into an `XmlTextReader`. In the current version of the .NET Framework, `XmlTextReader` has DTD processing enabled by default.

```
public class XMLHandler : IHttpHandler
{
  public void ProcessRequest(HttpContext ctx)
  {
      // BAD: XmlTextReader is insecure by default, and the payload is user-provided data
      XmlTextReader reader = new XmlTextReader(ctx.Request.QueryString["document"]);
  ...
}
}
```

The solution is to set the `DtdProcessing` property to `DtdProcessing.Prohibit`.

References

- OWASP: XML External Entity (XXE) Prevention Cheat Sheet.
- Microsoft Docs: System.XML: Security considerations.
- Common Weakness Enumeration: CWE-611.
- Common Weakness Enumeration: CWE-827.
- Common Weakness Enumeration: CWE-776.

High 8

Rule	Found on	Discovered by
Uncontrolled data used in path expression	2026-06-01T09:03:22Z	CodeQL
Uncontrolled data used in path expression	2026-06-01T09:03:22Z	CodeQL
Uncontrolled data used in path expression	2026-06-01T09:03:22Z	CodeQL
Uncontrolled data used in path expression	2026-06-01T09:03:22Z	CodeQL
SQL query built from user-controlled sources	2026-06-01T09:03:22Z	CodeQL
SQL query built from user-controlled sources	2026-06-01T09:03:22Z	CodeQL
SQL query built from user-controlled sources	2026-06-01T09:03:22Z	CodeQL
Encryption using ECB	2026-06-01T09:03:22Z	CodeQL

cs/path-injection

Location: VulnerableApi/Controllers/PathTraversalController.cs:32

Commit SHA: 90619238a29bec2d4f15d6bacdfa555db078de43

Message: This path depends on a user-provided value.

Description: Accessing paths influenced by users can allow an attacker to access unexpected resources.

Precision: high

Severity: error

cwe-022,cwe-023,cwe-036,cwe-073,cwe-099

Uncontrolled data used in path expression

Accessing paths controlled by users can allow an attacker to access unexpected resources. This can result in sensitive information being revealed or deleted, or an attacker being able to influence behavior by modifying unexpected files.

Paths that are naively constructed from data controlled by a user may be absolute paths, or may contain unexpected special characters such as "..". Such a path could point anywhere on the file system.

Recommendation

Validate user input before using it to construct a file path.

Common validation methods include checking that the normalized path is relative and does not contain any ".." components, or checking that the path is contained within a safe folder. The method you should use depends on how the path is used in the application, and whether the path should be a single path component.

If the path should be a single path component (such as a file name), you can check for the existence of any path separators ("/" or "\"), or ".." sequences in the input, and reject the input if any are found.

Note that removing "../" sequences is *not* sufficient, since the input could still contain a path separator followed by "..". For example, the input ".../...//" would still result in the string "../" if only "../" sequences are removed.

Finally, the simplest (but most restrictive) option is to use an allow list of safe patterns and make sure that the user input matches one of these patterns.

Example

In this example, a user-provided file name is read from a HTTP request and then used to access a file and send it back to the user. However, a malicious user could enter a file name anywhere on the file system, such as "/etc/passwd" or "../../../etc/passwd".

```
using System;
using System.IO;
using System.Web;

public class TaintedPathHandler : IHttpHandler
{
  public void ProcessRequest(HttpContext ctx)
  {
      string filename = ctx.Request.QueryString["path"];
      // BAD: This could read any file on the filesystem.
      ctx.Response.Write(File.ReadAllText(filename));
  }
}
```

If the input should only be a file name, you can check that it doesn't contain any path separators or ".." sequences.

```
using System;
using System.IO;
using System.Web;

public class TaintedPathHandler : IHttpHandler
{
  public void ProcessRequest(HttpContext ctx)
  {
      string filename = ctx.Request.QueryString["path"];
      // GOOD: ensure that the filename has no path separators or parent directory references
      if (filename.Contains("..") || filename.Contains("/") || filename.Contains("\\"))
      {
          ctx.Response.StatusCode = 400;
          ctx.Response.StatusDescription = "Bad Request";
          ctx.Response.Write("Invalid path");
          return;
      }
      ctx.Response.Write(File.ReadAllText(filename));
  }
}
```

If the input should be within a specific directory, you can check that the resolved path is still contained within that directory.

```
using System;
using System.IO;
using System.Web;

public class TaintedPathHandler : IHttpHandler
{
  public void ProcessRequest(HttpContext ctx)
  {
      string filename = ctx.Request.QueryString["path"];

      string user = ctx.User.Identity.Name;
      string publicFolder = Path.GetFullPath("/home/" + user + "/public");
      string filePath = Path.GetFullPath(Path.Combine(publicFolder, filename));

      // GOOD: ensure that the path stays within the public folder
      if (!filePath.StartsWith(publicFolder + Path.DirectorySeparatorChar))
      {
          ctx.Response.StatusCode = 400;
          ctx.Response.StatusDescription = "Bad Request";
          ctx.Response.Write("Invalid path");
          return;
      }
      ctx.Response.Write(File.ReadAllText(filename));
  }
}
```

References

- OWASP: Path Traversal.
- Common Weakness Enumeration: CWE-22.
- Common Weakness Enumeration: CWE-23.
- Common Weakness Enumeration: CWE-36.
- Common Weakness Enumeration: CWE-73.
- Common Weakness Enumeration: CWE-99.

cs/path-injection

Location: VulnerableApi/Controllers/PathTraversalController.cs:33

Commit SHA: 90619238a29bec2d4f15d6bacdfa555db078de43

Message: This path depends on a user-provided value.

Description: Accessing paths influenced by users can allow an attacker to access unexpected resources.

Precision: high

Severity: error

cwe-022,cwe-023,cwe-036,cwe-073,cwe-099

Uncontrolled data used in path expression

Accessing paths controlled by users can allow an attacker to access unexpected resources. This can result in sensitive information being revealed or deleted, or an attacker being able to influence behavior by modifying unexpected files.

Paths that are naively constructed from data controlled by a user may be absolute paths, or may contain unexpected special characters such as "..". Such a path could point anywhere on the file system.

Recommendation

Validate user input before using it to construct a file path.

Common validation methods include checking that the normalized path is relative and does not contain any ".." components, or checking that the path is contained within a safe folder. The method you should use depends on how the path is used in the application, and whether the path should be a single path component.

If the path should be a single path component (such as a file name), you can check for the existence of any path separators ("/" or "\"), or ".." sequences in the input, and reject the input if any are found.

Note that removing "../" sequences is *not* sufficient, since the input could still contain a path separator followed by "..". For example, the input ".../.../" would still result in the string "../" if only "../" sequences are removed.

Finally, the simplest (but most restrictive) option is to use an allow list of safe patterns and make sure that the user input matches one of these patterns.

Example

In this example, a user-provided file name is read from a HTTP request and then used to access a file and send it back to the user. However, a malicious user could enter a file name anywhere on the file system, such as "/etc/passwd" or "../../../etc/passwd".

```
using System;
using System.IO;
using System.Web;

public class TaintedPathHandler : IHttpHandler
{
  public void ProcessRequest(HttpContext ctx)
  {
      string filename = ctx.Request.QueryString["path"];
      // BAD: This could read any file on the filesystem.
      ctx.Response.Write(File.ReadAllText(filename));
  }
}
```

If the input should only be a file name, you can check that it doesn't contain any path separators or ".." sequences.

```
using System;
using System.IO;
using System.Web;

public class TaintedPathHandler : IHttpHandler
{
  public void ProcessRequest(HttpContext ctx)
  {
      string filename = ctx.Request.QueryString["path"];
      // GOOD: ensure that the filename has no path separators or parent directory references
      if (filename.Contains("..") || filename.Contains("/") || filename.Contains("\\"))
      {
          ctx.Response.StatusCode = 400;
          ctx.Response.StatusDescription = "Bad Request";
          ctx.Response.Write("Invalid path");
          return;
      }
      ctx.Response.Write(File.ReadAllText(filename));
  }
}
```

If the input should be within a specific directory, you can check that the resolved path is still contained within that directory.

```
using System;
using System.IO;
using System.Web;

public class TaintedPathHandler : IHttpHandler
{
  public void ProcessRequest(HttpContext ctx)
  {
      string filename = ctx.Request.QueryString["path"];

      string user = ctx.User.Identity.Name;
      string publicFolder = Path.GetFullPath("/home/" + user + "/public");
      string filePath = Path.GetFullPath(Path.Combine(publicFolder, filename));

      // GOOD: ensure that the path stays within the public folder
      if (!filePath.StartsWith(publicFolder + Path.DirectorySeparatorChar))
      {
          ctx.Response.StatusCode = 400;
          ctx.Response.StatusDescription = "Bad Request";
          ctx.Response.Write("Invalid path");
          return;
      }
      ctx.Response.Write(File.ReadAllText(filename));
  }
}
```

References

- OWASP: Path Traversal.
- Common Weakness Enumeration: CWE-22.
- Common Weakness Enumeration: CWE-23.
- Common Weakness Enumeration: CWE-36.
- Common Weakness Enumeration: CWE-73.
- Common Weakness Enumeration: CWE-99.

cs/path-injection

Location: VulnerableApi/Controllers/PathTraversalController.cs:23

Commit SHA: 90619238a29bec2d4f15d6bacdfa555db078de43

Message: This path depends on a user-provided value.

Description: Accessing paths influenced by users can allow an attacker to access unexpected resources.

Precision: high

Severity: error

cwe-022,cwe-023,cwe-036,cwe-073,cwe-099

Uncontrolled data used in path expression

Accessing paths controlled by users can allow an attacker to access unexpected resources. This can result in sensitive information being revealed or deleted, or an attacker being able to influence behavior by modifying unexpected files.

Paths that are naively constructed from data controlled by a user may be absolute paths, or may contain unexpected special characters such as "..". Such a path could point anywhere on the file system.

Recommendation

Validate user input before using it to construct a file path.

Common validation methods include checking that the normalized path is relative and does not contain any ".." components, or checking that the path is contained within a safe folder. The method you should use depends on how the path is used in the application, and whether the path should be a single path component.

If the path should be a single path component (such as a file name), you can check for the existence of any path separators ("/" or "\"), or ".." sequences in the input, and reject the input if any are found.

Note that removing "../" sequences is *not* sufficient, since the input could still contain a path separator followed by "..". For example, the input ".../.../" would still result in the string "../" if only "../" sequences are removed.

Finally, the simplest (but most restrictive) option is to use an allow list of safe patterns and make sure that the user input matches one of these patterns.

Example

In this example, a user-provided file name is read from a HTTP request and then used to access a file and send it back to the user. However, a malicious user could enter a file name anywhere on the file system, such as "/etc/passwd" or "../../../etc/passwd".

```
using System;
using System.IO;
using System.Web;

public class TaintedPathHandler : IHttpHandler
{
  public void ProcessRequest(HttpContext ctx)
  {
      string filename = ctx.Request.QueryString["path"];
      // BAD: This could read any file on the filesystem.
      ctx.Response.Write(File.ReadAllText(filename));
  }
}
```

If the input should only be a file name, you can check that it doesn't contain any path separators or ".." sequences.

```
using System;
using System.IO;
using System.Web;

public class TaintedPathHandler : IHttpHandler
{
  public void ProcessRequest(HttpContext ctx)
  {
      string filename = ctx.Request.QueryString["path"];
      // GOOD: ensure that the filename has no path separators or parent directory references
      if (filename.Contains("..") || filename.Contains("/") || filename.Contains("\\"))
      {
          ctx.Response.StatusCode = 400;
          ctx.Response.StatusDescription = "Bad Request";
          ctx.Response.Write("Invalid path");
          return;
      }
      ctx.Response.Write(File.ReadAllText(filename));
  }
}
```

If the input should be within a specific directory, you can check that the resolved path is still contained within that directory.

```
using System;
using System.IO;
using System.Web;

public class TaintedPathHandler : IHttpHandler
{
  public void ProcessRequest(HttpContext ctx)
  {
      string filename = ctx.Request.QueryString["path"];

      string user = ctx.User.Identity.Name;
      string publicFolder = Path.GetFullPath("/home/" + user + "/public");
      string filePath = Path.GetFullPath(Path.Combine(publicFolder, filename));

      // GOOD: ensure that the path stays within the public folder
      if (!filePath.StartsWith(publicFolder + Path.DirectorySeparatorChar))
      {
          ctx.Response.StatusCode = 400;
          ctx.Response.StatusDescription = "Bad Request";
          ctx.Response.Write("Invalid path");
          return;
      }
      ctx.Response.Write(File.ReadAllText(filename));
  }
}
```

References

- OWASP: Path Traversal.
- Common Weakness Enumeration: CWE-22.
- Common Weakness Enumeration: CWE-23.
- Common Weakness Enumeration: CWE-36.
- Common Weakness Enumeration: CWE-73.
- Common Weakness Enumeration: CWE-99.

cs/path-injection

Location: VulnerableApi/Controllers/PathTraversalController.cs:14

Commit SHA: 90619238a29bec2d4f15d6bacdfa555db078de43

Message: This path depends on a user-provided value.

Description: Accessing paths influenced by users can allow an attacker to access unexpected resources.

Precision: high

Severity: error

cwe-022,cwe-023,cwe-036,cwe-073,cwe-099

Uncontrolled data used in path expression

Accessing paths controlled by users can allow an attacker to access unexpected resources. This can result in sensitive information being revealed or deleted, or an attacker being able to influence behavior by modifying unexpected files.

Paths that are naively constructed from data controlled by a user may be absolute paths, or may contain unexpected special characters such as "..". Such a path could point anywhere on the file system.

Recommendation

Validate user input before using it to construct a file path.

Common validation methods include checking that the normalized path is relative and does not contain any ".." components, or checking that the path is contained within a safe folder. The method you should use depends on how the path is used in the application, and whether the path should be a single path component.

If the path should be a single path component (such as a file name), you can check for the existence of any path separators ("/" or "\"), or ".." sequences in the input, and reject the input if any are found.

Note that removing "../" sequences is *not* sufficient, since the input could still contain a path separator followed by "..". For example, the input ".../.../" would still result in the string "../" if only "../" sequences are removed.

Finally, the simplest (but most restrictive) option is to use an allow list of safe patterns and make sure that the user input matches one of these patterns.

Example

In this example, a user-provided file name is read from a HTTP request and then used to access a file and send it back to the user. However, a malicious user could enter a file name anywhere on the file system, such as "/etc/passwd" or "../../../etc/passwd".

```
using System;
using System.IO;
using System.Web;

public class TaintedPathHandler : IHttpHandler
{
  public void ProcessRequest(HttpContext ctx)
  {
      string filename = ctx.Request.QueryString["path"];
      // BAD: This could read any file on the filesystem.
      ctx.Response.Write(File.ReadAllText(filename));
  }
}
```

If the input should only be a file name, you can check that it doesn't contain any path separators or ".." sequences.

```
using System;
using System.IO;
using System.Web;

public class TaintedPathHandler : IHttpHandler
{
  public void ProcessRequest(HttpContext ctx)
  {
      string filename = ctx.Request.QueryString["path"];
      // GOOD: ensure that the filename has no path separators or parent directory references
      if (filename.Contains("..") || filename.Contains("/") || filename.Contains("\\"))
      {
          ctx.Response.StatusCode = 400;
          ctx.Response.StatusDescription = "Bad Request";
          ctx.Response.Write("Invalid path");
          return;
      }
      ctx.Response.Write(File.ReadAllText(filename));
  }
}
```

If the input should be within a specific directory, you can check that the resolved path is still contained within that directory.

```
using System;
using System.IO;
using System.Web;

public class TaintedPathHandler : IHttpHandler
{
  public void ProcessRequest(HttpContext ctx)
  {
      string filename = ctx.Request.QueryString["path"];

      string user = ctx.User.Identity.Name;
      string publicFolder = Path.GetFullPath("/home/" + user + "/public");
      string filePath = Path.GetFullPath(Path.Combine(publicFolder, filename));

      // GOOD: ensure that the path stays within the public folder
      if (!filePath.StartsWith(publicFolder + Path.DirectorySeparatorChar))
      {
          ctx.Response.StatusCode = 400;
          ctx.Response.StatusDescription = "Bad Request";
          ctx.Response.Write("Invalid path");
          return;
      }
      ctx.Response.Write(File.ReadAllText(filename));
  }
}
```

References

- OWASP: Path Traversal.
- Common Weakness Enumeration: CWE-22.
- Common Weakness Enumeration: CWE-23.
- Common Weakness Enumeration: CWE-36.
- Common Weakness Enumeration: CWE-73.
- Common Weakness Enumeration: CWE-99.

cs/sql-injection

Location: VulnerableApi/Controllers/SqlInjectionController.cs:60

Commit SHA: 90619238a29bec2d4f15d6bacdfa555db078de43

Message: This query depends on this ASP.NET Core MVC action method parameter.

Description: Building a SQL query from user-controlled sources is vulnerable to insertion of malicious SQL code by the user.

Precision: high

Severity: error

cwe-089

SQL query built from user-controlled sources

If a SQL query is built using string concatenation, and the components of the concatenation include user input, a user is likely to be able to run malicious database queries.

Recommendation

Usually, it is better to use a prepared statement than to build a complete query with string concatenation. A prepared statement can include a parameter, written as either a question mark (`?`) or with an explicit name (`@parameter`), for each part of the SQL query that is expected to be filled in by a different value each time it is run. When the query is later executed, a value must be supplied for each parameter in the query.

It is good practice to use prepared statements for supplying parameters to a query, whether or not any of the parameters are directly traceable to user input. Doing so avoids any need to worry about quoting and escaping.

Example

In the following example, the code runs a simple SQL query in three different ways.

The first way involves building a query, `query1`, by concatenating a user-supplied text box value with some string literals. The text box value can include special characters, so this code allows for SQL injection attacks.

The second way uses a stored procedure, `ItemsStoredProcedure`, with a single parameter (`@category`). The parameter is then given a value by calling `Parameters.Add`. This version is immune to injection attacks, because any special characters are not given any special treatment.

The third way builds a query, `query2`, with a single string literal that includes a parameter (`@category`). The parameter is then given a value by calling `Parameters.Add`. This version is immune to injection attacks, because any special characters are not given any special treatment.

```
using System.Data;
using System.Data.SqlClient;
using System.Web.UI.WebControls;

class SqlInjection
{
  TextBox categoryTextBox;
  string connectionString;

  public DataSet GetDataSetByCategory()
  {
      // BAD: the category might have SQL special characters in it
      using (var connection = new SqlConnection(connectionString))
      {
          var query1 = "SELECT ITEM,PRICE FROM PRODUCT WHERE ITEM_CATEGORY='"
            + categoryTextBox.Text + "' ORDER BY PRICE";
          var adapter = new SqlDataAdapter(query1, connection);
          var result = new DataSet();
          adapter.Fill(result);
          return result;
      }

      // GOOD: use parameters with stored procedures
      using (var connection = new SqlConnection(connectionString))
      {
          var adapter = new SqlDataAdapter("ItemsStoredProcedure", connection);
          adapter.SelectCommand.CommandType = CommandType.StoredProcedure;
          var parameter = new SqlParameter("category", categoryTextBox.Text);
          adapter.SelectCommand.Parameters.Add(parameter);
          var result = new DataSet();
          adapter.Fill(result);
          return result;
      }

      // GOOD: use parameters with dynamic SQL
      using (var connection = new SqlConnection(connectionString))
      {
          var query2 = "SELECT ITEM,PRICE FROM PRODUCT WHERE ITEM_CATEGORY="
            + "@category ORDER BY PRICE";
          var adapter = new SqlDataAdapter(query2, connection);
          var parameter = new SqlParameter("category", categoryTextBox.Text);
          adapter.SelectCommand.Parameters.Add(parameter);
          var result = new DataSet();
          adapter.Fill(result);
          return result;
      }
  }
}
```

References

- MSDN: How To: Protect From SQL Injection in ASP.NET.
- Common Weakness Enumeration: CWE-89.

cs/sql-injection

Location: VulnerableApi/Controllers/SqlInjectionController.cs:45

Commit SHA: 90619238a29bec2d4f15d6bacdfa555db078de43

Message: This query depends on this ASP.NET Core MVC action method parameter.

Description: Building a SQL query from user-controlled sources is vulnerable to insertion of malicious SQL code by the user.

Precision: high

Severity: error

cwe-089

SQL query built from user-controlled sources

If a SQL query is built using string concatenation, and the components of the concatenation include user input, a user is likely to be able to run malicious database queries.

Recommendation

Usually, it is better to use a prepared statement than to build a complete query with string concatenation. A prepared statement can include a parameter, written as either a question mark (`?`) or with an explicit name (`@parameter`), for each part of the SQL query that is expected to be filled in by a different value each time it is run. When the query is later executed, a value must be supplied for each parameter in the query.

It is good practice to use prepared statements for supplying parameters to a query, whether or not any of the parameters are directly traceable to user input. Doing so avoids any need to worry about quoting and escaping.

Example

In the following example, the code runs a simple SQL query in three different ways.

The first way involves building a query, `query1`, by concatenating a user-supplied text box value with some string literals. The text box value can include special characters, so this code allows for SQL injection attacks.

The second way uses a stored procedure, `ItemsStoredProcedure`, with a single parameter (`@category`). The parameter is then given a value by calling `Parameters.Add`. This version is immune to injection attacks, because any special characters are not given any special treatment.

The third way builds a query, `query2`, with a single string literal that includes a parameter (`@category`). The parameter is then given a value by calling `Parameters.Add`. This version is immune to injection attacks, because any special characters are not given any special treatment.

```
using System.Data;
using System.Data.SqlClient;
using System.Web.UI.WebControls;

class SqlInjection
{
  TextBox categoryTextBox;
  string connectionString;

  public DataSet GetDataSetByCategory()
  {
      // BAD: the category might have SQL special characters in it
      using (var connection = new SqlConnection(connectionString))
      {
          var query1 = "SELECT ITEM,PRICE FROM PRODUCT WHERE ITEM_CATEGORY='"
            + categoryTextBox.Text + "' ORDER BY PRICE";
          var adapter = new SqlDataAdapter(query1, connection);
          var result = new DataSet();
          adapter.Fill(result);
          return result;
      }

      // GOOD: use parameters with stored procedures
      using (var connection = new SqlConnection(connectionString))
      {
          var adapter = new SqlDataAdapter("ItemsStoredProcedure", connection);
          adapter.SelectCommand.CommandType = CommandType.StoredProcedure;
          var parameter = new SqlParameter("category", categoryTextBox.Text);
          adapter.SelectCommand.Parameters.Add(parameter);
          var result = new DataSet();
          adapter.Fill(result);
          return result;
      }

      // GOOD: use parameters with dynamic SQL
      using (var connection = new SqlConnection(connectionString))
      {
          var query2 = "SELECT ITEM,PRICE FROM PRODUCT WHERE ITEM_CATEGORY="
            + "@category ORDER BY PRICE";
          var adapter = new SqlDataAdapter(query2, connection);
          var parameter = new SqlParameter("category", categoryTextBox.Text);
          adapter.SelectCommand.Parameters.Add(parameter);
          var result = new DataSet();
          adapter.Fill(result);
          return result;
      }
  }
}
```

References

- MSDN: How To: Protect From SQL Injection in ASP.NET.

- Common Weakness Enumeration: CWE-89.

cs/sql-injection

Location: VulnerableApi/Controllers/SqlInjectionController.cs:26

Commit SHA: 90619238a29bec2d4f15d6bacdfa555db078de43

Message: This query depends on this ASP.NET Core MVC action method parameter.

Description: Building a SQL query from user-controlled sources is vulnerable to insertion of malicious SQL code by the user.

Precision: high

Severity: error

cwe-089

SQL query built from user-controlled sources

If a SQL query is built using string concatenation, and the components of the concatenation include user input, a user is likely to be able to run malicious database queries.

Recommendation

Usually, it is better to use a prepared statement than to build a complete query with string concatenation. A prepared statement can include a parameter, written as either a question mark (`?`) or with an explicit name (`@parameter`), for each part of the SQL query that is expected to be filled in by a different value each time it is run. When the query is later executed, a value must be supplied for each parameter in the query.

It is good practice to use prepared statements for supplying parameters to a query, whether or not any of the parameters are directly traceable to user input. Doing so avoids any need to worry about quoting and escaping.

Example

In the following example, the code runs a simple SQL query in three different ways.

The first way involves building a query, `query1`, by concatenating a user-supplied text box value with some string literals. The text box value can include special characters, so this code allows for SQL injection attacks.

The second way uses a stored procedure, `ItemsStoredProcedure`, with a single parameter (`@category`). The parameter is then given a value by calling `Parameters.Add`. This version is immune to injection attacks, because any special characters are not given any special treatment.

The third way builds a query, `query2`, with a single string literal that includes a parameter (`@category`). The parameter is then given a value by calling `Parameters.Add`. This version is immune to injection attacks, because any special characters are not given any special treatment.

```
using System.Data;
using System.Data.SqlClient;
using System.Web.UI.WebControls;

class SqlInjection
{
  TextBox categoryTextBox;
  string connectionString;

  public DataSet GetDataSetByCategory()
  {
      // BAD: the category might have SQL special characters in it
      using (var connection = new SqlConnection(connectionString))
      {
          var query1 = "SELECT ITEM,PRICE FROM PRODUCT WHERE ITEM_CATEGORY='"
            + categoryTextBox.Text + "' ORDER BY PRICE";
          var adapter = new SqlDataAdapter(query1, connection);
          var result = new DataSet();
          adapter.Fill(result);
          return result;
      }

      // GOOD: use parameters with stored procedures
      using (var connection = new SqlConnection(connectionString))
      {
          var adapter = new SqlDataAdapter("ItemsStoredProcedure", connection);
          adapter.SelectCommand.CommandType = CommandType.StoredProcedure;
          var parameter = new SqlParameter("category", categoryTextBox.Text);
          adapter.SelectCommand.Parameters.Add(parameter);
          var result = new DataSet();
          adapter.Fill(result);
          return result;
      }

      // GOOD: use parameters with dynamic SQL
      using (var connection = new SqlConnection(connectionString))
      {
          var query2 = "SELECT ITEM,PRICE FROM PRODUCT WHERE ITEM_CATEGORY="
            + "@category ORDER BY PRICE";
          var adapter = new SqlDataAdapter(query2, connection);
          var parameter = new SqlParameter("category", categoryTextBox.Text);
          adapter.SelectCommand.Parameters.Add(parameter);
          var result = new DataSet();
          adapter.Fill(result);
          return result;
      }
  }
}
```

References

- MSDN: How To: Protect From SQL Injection in ASP.NET.
- Common Weakness Enumeration: CWE-89.

cs/ecb-encryption

Location: VulnerableApi/Controllers/CryptoController.cs:52

Commit SHA: 90619238a29bec2d4f15d6bacdfa555db078de43

Message: The ECB (Electronic Code Book) encryption mode is vulnerable to replay attacks.

Description: Highlights uses of the encryption mode 'CipherMode.ECB'. This mode should normally not be used because it is vulnerable to replay attacks.

Precision: high

Severity: warning

cwe-327

Encryption using ECB

ECB should not be used as a mode for encryption. It has dangerous weaknesses. Data is encrypted the same way every time meaning the same plaintext input will always produce the same ciphertext. This makes encrypted messages vulnerable to replay attacks.

Recommendation

Use a different CypherMode.

References

- Wikipedia, Block cypher modes of operation, Electronic codebook (ECB).
- Common Weakness Enumeration: CWE-327.

Medium 3

Rule	Found on	Discovered by
URL redirection from remote source	2026-06-01T09:03:22Z	CodeQL
Log entries created from user input	2026-06-01T09:03:22Z	CodeQL
Missing XML validation	2026-06-01T09:03:22Z	CodeQL

cs/web/unvalidated-url-redirection

Location: VulnerableApi/Controllers/SsrfController.cs:23

Commit SHA: 90619238a29bec2d4f15d6bacdfa555db078de43

Message: Untrusted URL redirection due to user-provided value.

Description: URL redirection based on unvalidated user input may cause redirection to malicious web sites.

Precision: high

Severity: error

cwe-601

URL redirection from remote source

Directly incorporating user input into a URL redirect request without validating the input can facilitate phishing attacks. In these attacks, unsuspecting users can be redirected to a malicious site that looks very similar to the real site they intend to visit, but which is controlled by the attacker.

Recommendation

To guard against untrusted URL redirection, it is advisable to avoid putting user input directly into a redirect URL. Instead, maintain a list of authorized redirects on the server; then choose from that list based on the user input provided.

If this is not possible, then the user input should be validated in some other way, for example, by verifying that the target URL is on the same host as the current page.

Example

The following example shows an HTTP request parameter being used directly in a URL redirect without validating the input, which facilitates phishing attacks:

```
using System;
using System.Web;

public class UnvalidatedUrlHandler : IHttpHandler
{
  public void ProcessRequest(HttpContext ctx)
  {
      // BAD: a request parameter is incorporated without validation into a URL redirect
      ctx.Response.Redirect(ctx.Request.QueryString["page"]);
  }
}
```

One way to remedy the problem is to validate the user input against a known fixed string before doing the redirection:

```
using System;
using System.Web;
using System.Collections.Generic;

public class UnvalidatedUrlHandler : IHttpHandler
{
  private List<string> VALID_REDIRECTS = new List<string>{ "http://cwe.mitre.org/data/definitions/601.html",
"http://cwe.mitre.org/data/definitions/79.html" };

  public void ProcessRequest(HttpContext ctx)
  {
      if (VALID_REDIRECTS.Contains(ctx.Request.QueryString["page"]))
      {
          // GOOD: the request parameter is validated against a known list of strings
          ctx.Response.Redirect(ctx.Request.QueryString["page"]);
      }
  }
}
```

Alternatively, we can check that the target URL does not redirect to a different host by checking that the URL is either relative or on a known good host:

```
using System;
using System.Web;

public class UnvalidatedUrlHandler : IHttpHandler
{
  public void ProcessRequest(HttpContext ctx)
  {
      var urlString = ctx.Request.QueryString["page"];
      var url = new Uri(urlString, UriKind.RelativeOrAbsolute);

      var url = new Uri(redirectUrl, UriKind.RelativeOrAbsolute);
      if (!url.IsAbsoluteUri) {
          // GOOD: The redirect is to a relative URL
          ctx.Response.Redirect(url.ToString());
      }

      if (url.Host == "example.org") {
          // GOOD: The redirect is to a known host
          ctx.Response.Redirect(url.ToString());
      }
  }
}
```

Note that as written, the above code will allow redirects to URLs on `example.com`, which is harmless but perhaps not intended. You can substitute your own domain (if known) for `example.com` to prevent this.

References

- OWASP: Unvalidated Redirects and Forwards Cheat Sheet.
- Microsoft Docs: Preventing Open Redirection Attacks (C#).
- Common Weakness Enumeration: CWE-601.

cs/log-forging

Location: VulnerableApi/Controllers/XssController.cs:22

Commit SHA: 90619238a29bec2d4f15d6bacdfa555db078de43

Message: This log entry depends on a user-provided value.

Description: Building log entries from user-controlled sources is vulnerable to insertion of forged log entries by a malicious user.

Precision: high

Severity: error

cwe-117

Log entries created from user input

If unsanitized user input is written to a log entry, a malicious user may be able to forge new log entries.

Forgery can occur if a user provides some input with characters that are interpreted when the log output is displayed. If the log is displayed as a plain text file, then new line characters can be used by a malicious user. If the log is displayed as HTML, then arbitrary HTML may be include to spoof log entries.

Recommendation

User input should be suitably encoded before it is logged.

If the log entries are plain text then line breaks should be removed from user input, using `String.Replace` or similar. Care should also be taken that user input is clearly marked in log entries, and that a malicious user cannot cause confusion in other ways.

For log entries that will be displayed in HTML, user input should be HTML encoded using `HttpServerUtility.HtmlEncode` or similar before being logged, to prevent forgery and other forms of HTML injection.

Example

In the following example, a user name, provided by the user, is logged using a logging framework. In the first case, it is logged without any sanitization. In the second case, `String.Replace` is used to ensure no line endings are present in the user input.

```
using Microsoft.Extensions.Logging;
using System;
using System.IO;
using System.Web;

public class LogForgingHandler : IHttpHandler
{
  private ILogger logger;

  public void ProcessRequest(HttpContext ctx)
  {
      String username = ctx.Request.QueryString["username"];
      // BAD: User input logged as-is
      logger.Warn(username + " log in requested.");
      // GOOD: User input logged with new-lines removed
      logger.Warn(username.Replace(Environment.NewLine, "") + " log in requested");
  }
}
```

References

- OWASP: Log Injection.
- Common Weakness Enumeration: CWE-117.

cs/xml/missing-validation

Location: VulnerableApi/Controllers/XxeController.cs:21

Commit SHA: 90619238a29bec2d4f15d6bacdfa555db078de43

Message: This XML processing depends on a user-provided value without validation because the 'XmlReaderSettings' instance does not specify the 'ValidationType' as 'Schema'.

Description: User input should not be processed as XML without validating it against a known schema.

Precision: high

Severity: recommendation

cwe-112

Missing XML validation

If unsanitized user input is processed as XML, it should be validated against a known schema. If no validation occurs, or if the validation relies on the schema or DTD specified in the document itself, then the XML document may contain any data in any form, which may invalidate assumptions the program later makes.

Recommendation

All XML provided by a user should be validated against a known schema when it is processed.

If using `XmlReader.Create`, you should always pass an instance of `XmlReaderSettings`, with the following properties:

- `ValidationType` must be set to `Schema`. If this property is unset, no validation occurs. If it is set to `DTD`, the document is only validated against the DTD specified in the user-provided document itself - which could be specified as anything by a malicious user.
- `ValidationFlags` must not include `ProcessInlineSchema` or `ProcessSchemaLocation`. These flags allow a user to provide their own inline schema or schema location for validation, allowing a malicious user to bypass the known schema validation.

Example

In the following example, text provided by a user is loaded using `XmlReader.Create`. In the first three examples, insufficient validation occurs, because either no validation is specified, or validation is only specified against a DTD provided by the user, or the validation permits a user to provide an inline schema. In the final example, a known schema is provided, and validation is set, using an instance of `XmlReaderSettings`. This ensures that the user input is properly validated against the known schema.

```
using System;
using System.IO;
using System.Web;
using System.Xml;
using System.Xml.Schema;

public class MissingXmlValidationHandler : IHttpHandler
{

  public void ProcessRequest(HttpContext ctx)
  {
      String userProvidedXml = ctx.Request.QueryString["userProvidedXml"];

      // BAD: User provided XML is processed without any validation,
      //      because there is no settings instance configured.
      XmlReader.Create(new StringReader(userProvidedXml));

      // BAD: User provided XML is processed without any validation,
      //      because the settings instance specifies DTD as the ValidationType
      XmlReaderSettings badSettings = new XmlReaderSettings();
      badSettings.ValidationType = ValidationType.DTD;
      XmlReader.Create(new StringReader(userProvidedXml), badSettings);

      // BAD: User provided XML is processed with validation, but the ProcessInlineSchema
      //      option is specified, so an attacker can provide their own schema to validate
      //      against.
      XmlReaderSettings badInlineSettings = new XmlReaderSettings();
      badInlineSettings.ValidationType = ValidationType.Schema;
      badInlineSettings.ValidationFlags |= XmlSchemaValidationFlags.ProcessInlineSchema;
      XmlReader.Create(new StringReader(userProvidedXml), badInlineSettings);

      // GOOD: User provided XML is processed with validation
      XmlReaderSettings goodSettings = new XmlReaderSettings();
      goodSettings.ValidationType = ValidationType.Schema;
      goodSettings.Schemas = new XmlSchemaSet() { { "urn:my-schema", "my.xsd" } };
      XmlReader.Create(new StringReader(userProvidedXml), goodSettings);
  }
}
```

References

- Microsoft: XML Schema (XSD) Validation with XmlSchemaSet.
- Common Weakness Enumeration: CWE-112.

Note 3

Rule	Found on	Discovered by
Generic catch clause	2026-06-01T09:03:22Z	CodeQL
Call to 'System.IO.Path.Combine' may silently drop its earlier arguments	2026-06-01T09:03:22Z	CodeQL
Call to 'System.IO.Path.Combine' may silently drop its earlier arguments	2026-06-01T09:03:22Z	CodeQL

cs/catch-of-all-exceptions

Location: VulnerableApi/Controllers/SqlInjectionController.cs:64

Commit SHA: 90619238a29bec2d4f15d6bacdfa555db078de43

Message: Generic catch clause.

Description: Catching all exceptions with a generic catch clause may be overly broad, which can make errors harder to diagnose.

Precision: high

Severity: recommendation

cwe-396

Generic catch clause

Catching all exceptions with a generic catch clause may be overly broad. This can make errors harder to diagnose when exceptions are caught unintentionally.

Recommendation

If possible, catch only specific exception types to avoid catching unintended exceptions.

Example

In the following example, a division by zero is incorrectly handled by catching all exceptions.

```
double reciprocal(double input)
{
  try
  {
      return 1 / input;
  }
  catch
  {
      // division by zero, return 0
      return 0;
  }
}
```

In the corrected example, division by zero is correctly handled by only catching appropriate `DivideByZeroException` exceptions. Moreover, arithmetic overflow is now handled separately from division by zero by explicitly catching `OverflowException` exceptions.

```
double reciprocal(double input)
{
  try
  {
      return 1 / input;
  }
  catch (DivideByZeroException)
  {
      return 0;
  }
  catch (OverflowException)
  {
      return double.MaxValue;
  }
}
```

References

- Common Weakness Enumeration: CWE-396.

cs/path-combine

Location: VulnerableApi/Controllers/PathTraversalController.cs:31

Commit SHA: 90619238a29bec2d4f15d6bacdfa555db078de43

Message: Call to 'System.IO.Path.Combine' may silently drop its earlier arguments.

Description: 'Path.Combine' may silently drop its earlier arguments if its later arguments are absolute paths.

Precision: very-high

Severity: recommendation

Call to 'System.IO.Path.Combine' may silently drop its earlier arguments

`Path.Combine` may silently drop its earlier arguments if its later arguments are absolute paths. E.g. `Path.Combine("C:\\Users\\Me\\Documents", "C:\\Program Files\\") == "C:\\Program Files"`.

Recommendation

Use `Path.Join` instead.

References

- Microsoft Learn, .NET API browser, Path.Combine.
- Microsoft Learn, .NET API browser, Path.Join.

cs/path-combine

Location: VulnerableApi/Controllers/PathTraversalController.cs:13

Commit SHA: 90619238a29bec2d4f15d6bacdfa555db078de43

Message: Call to 'System.IO.Path.Combine' may silently drop its earlier arguments.

Description: 'Path.Combine' may silently drop its earlier arguments if its later arguments are absolute paths.

Precision: very-high

Severity: recommendation

Call to 'System.IO.Path.Combine' may silently drop its earlier arguments

`Path.Combine` may silently drop its earlier arguments if its later arguments are absolute paths. E.g. `Path.Combine("C:\\Users\\Me\\Documents", "C:\\Program Files\\") == "C:\\Program Files"`.

Recommendation

Use `Path.Join` instead.

References

- Microsoft Learn, .NET API browser, Path.Combine.
- Microsoft Learn, .NET API browser, Path.Join.

Dispositioned Vulnerabilities